

Linux Services Configuration Guide

NFS • DNS (BIND9) • Samba • SMTP • FTP • SFTP • RSyslog

Step-by-step setup for Ubuntu / RHEL servers

1	NFS — Network File System	Server + Client + Exports + Firewall
2	DNS — BIND9 Name Server	Zones + Forward/Reverse + Secondary
3	Samba — SMB/CIFS File Sharing	Shares + Users + SELinux + Client
4	SMTP — Postfix Mail Server	Install + Config + TLS + Testing
5	FTP — vsftpd File Transfer	Passive + Chroot + SSL/TLS
6	SFTP — SSH File Transfer	Subsystem + Chroot Jail + Keys
7	RSyslog — Centralized Logging	Server + Client + Templates + Filters

1 NFS — Network File System

NFS allows Linux machines to share directories over the network. The server exports directories; clients mount them as local paths.

■■ SERVER SETUP ■■

Step 1: Install NFS Server

```
# Ubuntu / Debian
sudo apt install nfs-kernel-server nfs-common -y

# RHEL / CentOS / Fedora
sudo dnf install nfs-utils -y
```

Step 2: Create Export Directory & Set Permissions

```
sudo mkdir -p /srv/nfs/shared
sudo chown nobody:nogroup /srv/nfs/shared # Ubuntu
sudo chown nfsnobody:nfsnobody /srv/nfs/shared # RHEL
sudo chmod 755 /srv/nfs/shared
```

Step 3: Configure /etc/exports

```
sudo nano /etc/exports

# Syntax: /path client(options)

# Single host – read/write
/srv/nfs/shared 192.168.1.100(rw,sync,no_subtree_check)

# Entire subnet – read/write
/srv/nfs/shared 192.168.1.0/24(rw,sync,no_root_squash,no_subtree_check)

# Read-only for all
/srv/nfs/shared *(ro,sync,no_subtree_check)
```

Common export options:

Option	Description
rw	Read and write access
ro	Read-only access
sync	Write data to disk before replying
async	Reply before writing (faster, less safe)
no_subtree_check	Improves reliability; disables subtree checking
no_root_squash	Remote root keeps root privileges
root_squash	Maps remote root to anonymous (safer default)
all_squash	Maps all users to anonymous
anonuid=1001	UID used for anonymous mapping
anongid=1001	GID used for anonymous mapping

Step 4: Apply Exports & Start Services

```
sudo exportfs -arv # Apply all exports (-a all, -r re-export, -v verbose)
sudo exportfs -v # Verify what is currently exported

# Start + enable
sudo systemctl enable --now nfs-kernel-server # Ubuntu
sudo systemctl enable --now nfs-server # RHEL
sudo systemctl enable --now rpcbind
```

Step 5: Open Firewall on Server

```
# UFW (Ubuntu)
sudo ufw allow from 192.168.1.0/24 to any port 111
sudo ufw allow from 192.168.1.0/24 to any port 2049
sudo ufw reload

# Firewallld (RHEL)
sudo firewall-cmd --permanent --add-service=nfs
sudo firewall-cmd --permanent --add-service=rpc-bind
sudo firewall-cmd --permanent --add-service=mountd
sudo firewall-cmd --reload
```

■■ CLIENT SETUP ■■

Step 1: Install NFS Client

```
sudo apt install nfs-common -y # Ubuntu
sudo dnf install nfs-utils -y # RHEL
```

Step 2: Discover Available Shares

```
showmount -e <SERVER-IP> # List exports from server
rpcinfo -p <SERVER-IP> # List all RPC services
```

Step 3: Mount Manually (Temporary)

```
sudo mkdir -p /mnt/nfs/shared
sudo mount -t nfs <SERVER-IP>:/srv/nfs/shared /mnt/nfs/shared

# Verify
df -h | grep nfs
mount | grep nfs
```

Step 4: Persistent Mount via /etc/fstab

```
sudo nano /etc/fstab

<SERVER-IP>:/srv/nfs/shared /mnt/nfs/shared nfs defaults,_netdev,rw,soft,timeo=30 0 0

sudo mount -a # Apply fstab
```

Key fstab options:

Option	Description
_netdev	Wait for network before mounting (prevents boot hang)
soft	Return error if server not reachable
hard	Retry forever until server responds (default)
timeo=30	Timeout in tenths of a second
nfsvers=4	Force NFS version 4

rsize/wsize	Read/Write block size (e.g. 8192)
-------------	-----------------------------------

■■ TROUBLESHOOTING ■■

```
sudo exportfs -v # Show active exports
rpcinfo -p <SERVER-IP> # Check RPC portmapper
showmount -e <SERVER-IP> # List available exports
nfsstat -s # Server statistics
nfsstat -c # Client statistics
sudo journalctl -u nfs-server -f # Live server logs
```

2 DNS — BIND9 Name Server

BIND9 provides DNS resolution — translating hostnames to IPs (forward) and IPs to hostnames (reverse). It can act as authoritative server, recursive resolver, or forwarder.

■■ SERVER SETUP ■■

Step 1: Install BIND9

```
# Ubuntu
sudo apt install bind9 bind9utils bind9-doc dnsutils -y

# RHEL
sudo dnf install bind bind-utils -y
```

Step 2: Understand Configuration Structure

```
/etc/bind/ # Ubuntu config root
named.conf # Includes the three files below
named.conf.options # Global options (forwarders, recursion)
named.conf.local # Your zone declarations
named.conf.default-zones # Default zones (localhost etc.)
/etc/bind/zones/ # Zone data files (you create this)

/etc/named.conf # RHEL single config file
```

Step 3: Configure named.conf.options (Global Options)

```
sudo nano /etc/bind/named.conf.options

options {
    directory "/var/cache/bind";

    # Who can query this server
    allow-query { localhost; 192.168.1.0/24; };

    # Who can use recursion
    allow-recursion { localhost; 192.168.1.0/24; };

    # Forward unresolved queries upstream
    forwarders { 8.8.8.8; 8.8.4.4; 1.1.1.1; };
    forward only;

    dnssec-validation auto;
    auth-nxdomain no;
    listen-on { any; };
    recursion yes;
    version "not disclosed"; # Hide BIND version
};
```

Step 4: Declare Zones in named.conf.local

```
sudo nano /etc/bind/named.conf.local

# ■■ Forward Zone ■■
zone "example.local" IN {
```

```
type master;
file "/etc/bind/zones/db.example.local";
allow-update { none; };
allow-transfer { 192.168.1.124; }; # Secondary server IP
};
```

```
# ■■ Reverse Zone (subnet 192.168.1.0/24) ■■
zone "1.168.192.in-addr.arpa" IN {
type master;
file "/etc/bind/zones/db.192.168.1";
allow-update { none; };
allow-transfer { 192.168.1.124; };
};
```

■ Reverse zone name: reverse the first three octets of the subnet. 192.168.1.0/24 → 1.168.192.in-addr.arpa

Step 5: Create Forward Zone File

```
sudo mkdir -p /etc/bind/zones
sudo nano /etc/bind/zones/db.example.local

$TTL 604800
@ IN SOA ns1.example.local. admin.example.local. (
2024010101 ; Serial ← MUST increment on every change
3600 ; Refresh
1800 ; Retry
604800 ; Expire
86400 ) ; Negative Cache TTL
```

```
; ■■ Name Servers ■■
@ IN NS ns1.example.local.
@ IN NS ns2.example.local.
```

```
; ■■ A Records (hostname → IP) ■■
ns1 IN A 192.168.1.123
ns2 IN A 192.168.1.124
server1 IN A 192.168.1.10
client1 IN A 192.168.1.100
web IN A 192.168.1.200
```

```
; ■■ CNAME Records (alias) ■■
www IN CNAME web.example.local.
nfs IN CNAME server1.example.local.
```

```
; ■■ MX Record (Mail) ■■
@ IN MX 10 mail.example.local.
mail IN A 192.168.1.202
```

Step 6: Create Reverse Zone File

```
sudo nano /etc/bind/zones/db.192.168.1

$TTL 604800
@ IN SOA ns1.example.local. admin.example.local. (
2024010101 3600 1800 604800 86400 )
```

```
@ IN NS ns1.example.local.
@ IN NS ns2.example.local.

; ■■ PTR Records (last octet → hostname) ■■
123 IN PTR ns1.example.local.
124 IN PTR ns2.example.local.
10 IN PTR server1.example.local.
100 IN PTR client1.example.local.
200 IN PTR web.example.local.
```

Step 7: Validate, Set Permissions & Start

```
# Validate syntax
sudo named-checkconf
sudo named-checkzone example.local /etc/bind/zones/db.example.local
sudo named-checkzone 1.168.192.in-addr.arpa /etc/bind/zones/db.192.168.1

# Each should print: OK

# Set permissions
sudo chown -R bind:bind /etc/bind/zones
sudo chmod 755 /etc/bind/zones
sudo chmod 644 /etc/bind/zones/*

# Start BIND9
sudo systemctl enable --now bind9 # Ubuntu
sudo systemctl enable --now named # RHEL

# Firewall
sudo ufw allow 53/tcp && sudo ufw allow 53/udp && sudo ufw reload # Ubuntu
sudo firewall-cmd --permanent --add-service=dns && sudo firewall-cmd --reload
```

■■ CLIENT CONFIGURATION ■■

Step 1: Set DNS on Client (Ubuntu — Netplan)

```
sudo nano /etc/netplan/00-installer-config.yaml

network:
  ethernets:
    ens33:
      dhcp4: false
      addresses: [192.168.1.100/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [192.168.1.123, 192.168.1.124]
      search: [example.local]
  version: 2

sudo netplan apply
```

Step 2: Test DNS Resolution

```
nslookup server1.example.local 192.168.1.123
dig server1.example.local @192.168.1.123
dig -x 192.168.1.10 @192.168.1.123 # Reverse lookup
```

```
dig MX example.local @192.168.1.123 # MX record
host server1.example.local
ping server1.example.local
```

■■ SECONDARY DNS SERVER ■■

```
# On secondary server (192.168.1.124)
sudo nano /etc/bind/named.conf.local

zone "example.local" IN {
type slave;
masters { 192.168.1.123; };
file "/var/cache/bind/db.example.local";
};

zone "1.168.192.in-addr.arpa" IN {
type slave;
masters { 192.168.1.123; };
file "/var/cache/bind/db.192.168.1";
};

sudo systemctl restart bind9
# Zone files transfer automatically from primary
```


3 Samba — SMB/CIFS File Sharing

Samba provides SMB/CIFS file and print sharing compatible with Windows, macOS, and Linux clients.

■ ■ SERVER SETUP ■ ■

Step 1: Install Samba

```
sudo apt install samba samba-common-bin smbclient -y # Ubuntu
sudo dnf install samba samba-common samba-client -y # RHEL
```

Step 2: Backup & Configure smb.conf

```
workgroup = WORKGROUP
```

```
server string = Samba Server %v
```

[illegible]

```
# ■■ Private Share (authenticated users) ■■■■■■■■■■■■■■■■■■
```

[illegible]

```
writable = yes
valid users = %S
create mask = 0700
directory mask = 0700
```

Step 3: Create Directories & Groups

```
# Public share
sudo mkdir -p /srv/samba/public
sudo chmod 0775 /srv/samba/public
sudo chown nobody:nogroup /srv/samba/public

# Private share
sudo mkdir -p /srv/samba/private
sudo groupadd sambagroup
sudo chown root:sambagroup /srv/samba/private
sudo chmod 0770 /srv/samba/private
```

Step 4: Create Samba Users

```
# Create OS user first (no login shell)
sudo useradd -M -s /sbin/nologin sambauser1
sudo usermod -aG sambagroup sambauser1

# Add to Samba password database
sudo smbpasswd -a sambauser1 # Set Samba password
sudo smbpasswd -e sambauser1 # Enable user

# Other user management
sudo smbpasswd -d sambauser1 # Disable
sudo smbpasswd -x sambauser1 # Delete
pdbedit -L -v # List all Samba users
```

Step 5: SELinux (RHEL Only)

```
sudo setsebool -P samba_enable_home_dirs on
sudo setsebool -P samba_export_all_rw on
sudo semanage fcontext -a -t samba_share_t "/srv/samba(/.*)?"
sudo restorecon -Rv /srv/samba
```

Step 6: Validate Config & Start Services

```
testparm # Validate smb.conf
testparm -s # Show effective config

sudo systemctl enable --now smbd nmbd # Ubuntu (smbd=file, nmbd=names)
sudo systemctl enable --now smb nmb # RHEL

# Firewall
sudo ufw allow samba && sudo ufw reload # Ubuntu
sudo firewall-cmd --permanent --add-service=samba && sudo firewall-cmd --reload
```

■■ CLIENT SETUP ■■

Step 1: Install Client Tools

```
sudo apt install smbclient cifs-utils -y # Ubuntu
sudo dnf install samba-client cifs-utils -y # RHEL
```

Step 2: Create Credentials File (Secure)

```
sudo nano /etc/samba/credentials

username=sambauser1
password=yourpassword
domain=WORKGROUP

sudo chmod 600 /etc/samba/credentials
sudo chown root:root /etc/samba/credentials
```

Step 3: Mount & Persist via /etc/fstab

```
sudo mkdir -p /mnt/samba/private

# Manual mount
sudo mount -t cifs //192.168.1.10/private /mnt/samba/private \
-o credentials=/etc/samba/credentials,uid=1000,gid=1000

# /etc/fstab (persistent)
//192.168.1.10/private /mnt/samba/private cifs
credentials=/etc/samba/credentials,_netdev,uid=1000,gid=1000,vers=3.0 0 0

sudo mount -a
```

■■ TROUBLESHOOTING ■■

```
testparm # Check smb.conf syntax
smbstatus # Show active connections
smbclient -L //server -U user # List shares
nmblookup -A <server-ip> # NetBIOS lookup
sudo journalctl -u smbd -f # Live logs
```

Postfix is the most widely used SMTP server on Linux. It handles sending, receiving, and relaying of email.

■■ SERVER SETUP ■■

Step 1: Install Postfix

Step 2: Configure /etc/postfix/main.cf

[illegible]

Step 3: Configure Relay via Gmail (Optional)

```
# In main.cf
relayhost = [smtp.gmail.com]:587
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options = noanonymous
smtp_tls_security_level = encrypt

# Create credentials file
sudo nano /etc/postfix/sasl_passwd
```

```
[smtp.gmail.com]:587 youruser@gmail.com:your-app-password
```

```
sudo postmap /etc/postfix/sasl_passwd
```

```
sudo chmod 600 /etc/postfix/sasl_passwd
```

Step 4: Enable TLS (Secure SMTP)

```
# Generate self-signed cert (or use Let's Encrypt)
```

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
```

```
-keyout /etc/ssl/private/mail.key \
```

```
-out /etc/ssl/certs/mail.crt
```

```
# Add to main.cf
```

```
smtpd_tls_cert_file = /etc/ssl/certs/mail.crt
```

```
smtpd_tls_key_file = /etc/ssl/private/mail.key
```

```
smtpd_use_tls = yes
```

```
smtpd_tls_session_cache_database = btree:${data_directory}/smtpd_scache
```

```
smtp_tls_session_cache_database = btree:${data_directory}/smtp_scache
```

```
smtpd_tls_security_level = may # 'encrypt' to force TLS
```

```
smtp_tls_security_level = may
```

Step 5: Configure master.cf (Ports 465/587)

```
sudo nano /etc/postfix/master.cf
```

```
# Uncomment submission (port 587) and smtps (port 465)
```

```
submission inet n - y - - smtpd
```

```
-o smtpd_tls_security_level=encrypt
```

```
-o smtpd_sasl_auth_enable=yes
```

```
-o smtpd_client_restrictions=permit_sasl_authenticated,reject
```

```
smtps inet n - y - - smtpd
```

```
-o smtpd_tls_wrappermode=yes
```

```
-o smtpd_sasl_auth_enable=yes
```

Step 6: Start Postfix & Configure Firewall

```
sudo systemctl enable --now postfix
```

```
sudo systemctl status postfix
```

```
# Firewall
```

```
sudo ufw allow 25/tcp # SMTP
```

```
sudo ufw allow 465/tcp # SMTPS
```

```
sudo ufw allow 587/tcp # Submission
```

```
sudo ufw reload
```

Step 7: Test SMTP

```
# Send test email via command line
```

```
echo "Test body" | mail -s "Test Subject" user@example.local
```

```
# Manual SMTP session
```

```
telnet localhost 25
```

```
EHLO example.local
```

```
MAIL FROM:<sender@example.local>
```

```
RCPT TO:<recipient@example.local>
```

```
DATA
```

```
Subject: Test
Hello world
.
QUIT

# Check mail queue
mailq
sudo postfix -p

# Force queue flush
sudo postfix -f

# View mail logs
sudo tail -f /var/log/mail.log
```

5 FTP — vsftpd File Transfer Protocol

vsftpd (Very Secure FTP Daemon) is the standard FTP server for Linux. It supports anonymous access, local users, chroot jails, and SSL/TLS.

■■ SERVER SETUP ■■

Step 1: Install vsftpd

```
sudo apt install vsftpd -y # Ubuntu
sudo dnf install vsftpd -y # RHEL
```

Step 2: Backup & Configure vsftpd.conf

```
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.bak
sudo nano /etc/vsftpd.conf
# █ Core Settings ████████████████████████████████████████████
listen=YES # Standalone mode
listen_ipv6=NO # Disable IPv6 listener

# █ Access Control ████████████████████████████████████████████
anonymous_enable=NO # No anonymous login
local_enable=YES # Allow local OS users
write_enable=YES # Allow upload/delete
local_umask=022 # Default file permissions

# █ Chroot Jail (restrict users to home dir) ██████████
chroot_local_user=YES # Jail all users
allow_writeable_chroot=YES # Allow writes in chroot
chroot_list_enable=NO # Disable per-user exceptions

# █ Passive Mode (important for firewalls/NAT) ████████
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=50000
pasv_address=192.168.1.10 # Server's public IP

# █ Logging ████████████████████████████████████████████████████████████
xferlog_enable=YES
xferlog_file=/var/log/vsftpd.log
xferlog_std_format=YES
log_ftp_protocol=YES # Verbose protocol logging

# █ Banner ████████████████████████████████████████████████████████████
ftpd_banner>Welcome to FTP Server

# █ User List ████████████████████████████████████████████████████████████
userlist_enable=YES
userlist_file=/etc/vsftpd.userlist
userlist_deny=NO # NO = only listed users allowed
```

Step 3: Create FTP User & User List

```
# Create FTP user
sudo useradd -m ftpuser1
```

```
sudo passwd ftpuser1

# Allow only listed users
echo 'ftpuser1' | sudo tee -a /etc/vsftpd.userlist
```

Step 4: Enable SSL/TLS (Secure FTP)

```
# Generate certificate
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/vsftpd.key \
-out /etc/ssl/certs/vsftpd.crt

# Add to vsftpd.conf
ssl_enable=YES
rsa_cert_file=/etc/ssl/certs/vsftpd.crt
rsa_private_key_file=/etc/ssl/private/vsftpd.key
ssl_tlsv1=YES
ssl_sslv2=NO
ssl_sslv3=NO
require_ssl_reuse=NO
ssl_ciphers=HIGH
force_local_data_ssl=YES # Encrypt data transfers
force_local_logins_ssl=YES # Encrypt logins
```

Step 5: Start vsftpd & Configure Firewall

```
sudo systemctl enable --now vsftpd
sudo systemctl status vsftpd

# Firewall
sudo ufw allow 20/tcp # FTP data
sudo ufw allow 21/tcp # FTP control
sudo ufw allow 40000:50000/tcp # Passive mode range
sudo ufw reload
```

Step 6: Test FTP Connection

```
# Test from client
ftp 192.168.1.10

# Or use lftp (better client)
sudo apt install lftp -y
lftp -u ftpuser1 192.168.1.10
ls
get remotefile.txt
put localfile.txt
bye

# View FTP logs
sudo tail -f /var/log/vsftpd.log
```


6 SFTP — SSH File Transfer Protocol

SFTP runs over SSH (port 22) and provides encrypted file transfer. It requires no extra installation — it's built into OpenSSH. A chroot jail restricts users to their home directories.

■■ SERVER SETUP ■■

Step 1: Install OpenSSH Server

```
sudo apt install openssh-server -y # Ubuntu
sudo dnf install openssh-server -y # RHEL

sudo systemctl enable --now sshd
```

Step 2: Configure SFTP in sshd_config

```
sudo nano /etc/ssh/sshd_config

# ■■ Ensure internal-sftp subsystem is set ■■■■■■■■■■■■
# Comment out or remove the existing Subsystem line:
# Subsystem sftp /usr/lib/openssh/sftp-server

# Replace with:
Subsystem sftp internal-sftp

# ■■ Chroot Jail for SFTP group ■■■■■■■■■■■■■■■■■■■■■■
Match Group sftpusers
ChrootDirectory /srv/sftp/%u # %u = username
ForceCommand internal-sftp
AllowTcpForwarding no
X11Forwarding no
PasswordAuthentication yes

■■ ChrootDirectory MUST be owned by root:root with permissions 755. Otherwise SSH will reject the chroot.
```

Step 3: Create SFTP Users & Directory Structure

```
# Create the SFTP group
sudo groupadd sftpusers

# Create user
sudo useradd -m -s /sbin/nologin -G sftpusers sftpuser1
sudo passwd sftpuser1

# Create chroot directory (owned by root – CRITICAL)
sudo mkdir -p /srv/sftp/sftpuser1
sudo chown root:root /srv/sftp/sftpuser1
sudo chmod 755 /srv/sftp/sftpuser1

# Create writable upload dir inside chroot
sudo mkdir -p /srv/sftp/sftpuser1/uploads
sudo chown sftpuser1:sftpusers /srv/sftp/sftpuser1/uploads
sudo chmod 755 /srv/sftp/sftpuser1/uploads
```

Step 4: Restart SSH & Configure Firewall

```
sudo sshd -t # Test config before restart
```

```
sudo systemctl restart sshd

# Firewall (SFTP uses port 22)
sudo ufw allow 22/tcp && sudo ufw reload
sudo firewall-cmd --permanent --add-service=ssh && sudo firewall-cmd --reload
```

■■ CLIENT USAGE ■■

Step 1: Connect via SFTP

```
# Basic connection
sftp sftpuser1@192.168.1.10

# SFTP commands
sftp> ls # List remote files
sftp> lls # List local files
sftp> pwd # Remote working dir
sftp> lpwd # Local working dir
sftp> get remotefile.txt # Download
sftp> put localfile.txt # Upload
sftp> mkdir newdir # Create remote dir
sftp> rm file.txt # Delete remote file
sftp> bye # Exit

# Non-interactive (single file)
sftp sftpuser1@192.168.1.10:/uploads/report.txt /local/path/
```

Step 2: SSH Key-Based Authentication (Passwordless)

```
# On CLIENT: Generate key pair
ssh-keygen -t ed25519 -C 'sftp-client'

# Copy public key to server
ssh-copy-id sftpuser1@192.168.1.10

# Or manually:
cat ~/.ssh/id_ed25519.pub | ssh sftpuser1@192.168.1.10 'mkdir -p ~/.ssh && cat >>
~/.ssh/authorized_keys'

# Secure key files on server
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys

# Disable password auth (after keys are set) in sshd_config:
PasswordAuthentication no
```

Step 3: SCP — Secure Copy (Quick Alternative)

```
# Copy local → remote
scp /local/file.txt sftpuser1@192.168.1.10:/uploads/

# Copy remote → local
scp sftpuser1@192.168.1.10:/uploads/file.txt /local/path/

# Copy directory recursively
scp -r /local/dir/ sftpuser1@192.168.1.10:/uploads/
```

7 RSyslog — Centralized Log Management

RSyslog collects logs from multiple servers/clients and centralizes them on one log server. Clients forward logs via TCP (reliable) or UDP (faster). The server stores logs per-host or per-service.

■ ■ SERVER SETUP (Log Collector) ■ ■

Step 1: Install RSyslog

```
sudo apt install rsyslog -y # Ubuntu
sudo dnf install rsyslog -y # RHEL
```

Step 2: Configure RSyslog Server (/etc/rsyslog.conf)

[illegible]

Step 3: Create Log Directory & Set Permissions

```
sudo mkdir -p /var/log/remote
sudo chown syslog:adm /var/log/remote
sudo chmod 755 /var/log/remote
```

Step 4: Advanced Template — Separate Files by Facility

Step 5: Open Firewall & Start RSyslog Server

```
sudo systemctl restart rsyslog
sudo systemctl status rsyslog

# Verify it's listening
ss -tulnp | grep 514
netstat -tulnp | grep 514
```

■■ CLIENT SETUP (Log Forwarder) ■■

Step 1: Install RSyslog on Client

```
sudo apt install rsyslog -y
```

Step 2: Configure Client to Forward Logs

```
sudo nano /etc/rsyslog.d/10-forward.conf

# Forward ALL logs to server via TCP (recommended)
*. * @@192.168.1.50:514
# @@ = TCP (reliable) @ = UDP (faster, less reliable)

# Forward only specific facilities
auth,authpriv.* @@192.168.1.50:514 # Auth logs only
kern.* @@192.168.1.50:514 # Kernel logs only
*.err @@192.168.1.50:514 # All errors

# Queue failed messages (retry when server is back)
*. * action(type="omfwd"
target="192.168.1.50"
port="514"
protocol="tcp"
action.resumeRetryCount="100"
queue.type="LinkedList"
queue.size="10000")
```

Step 3: Restart Client RSyslog

```
sudo systemctl restart rsyslog
sudo systemctl status rsyslog

# Test – generate a log entry
logger -t TEST 'This is a test log from client1'

# Verify on SERVER
ls /var/log/remote/
tail -f /var/log/remote/<client-hostname>/TEST.log
```

■■ LOG ROTATION ■■

Step 1: Configure logrotate for Remote Logs

```
sudo nano /etc/logrotate.d/rsyslog-remote

/var/log/remote/*/*.log {
daily # Rotate daily
rotate 30 # Keep 30 days
compress # Gzip old logs
delaycompress # Compress after 1 rotation
```

```
missingok # OK if file missing
notifempty # Don't rotate empty files
sharedscripts
postrotate
/usr/lib/rsyslog/rsyslog-rotate # Signal rsyslog
endscript
}
```

■■ WINDOWS CLIENT FORWARDING (NXLog) ■■

```
# Install NXLog CE on Windows from: nxlog.co/products/nxlog-community-edition

# Edit: C:\Program Files (x86)\nxlog\conf\nxlog.conf

<Input eventlog>
Module im_msvistalog
Query <QueryList>...
</Input>

<Output syslog_out>
Module om_tcp
Host 192.168.1.50
Port 514
Exec to_syslog_snare();
</Output>

<Route 1>
Path eventlog => syslog_out
</Route>

# Restart NXLog service in Windows
net stop nxlog && net start nxlog
```

■■ USEFUL RSYSLOG COMMANDS ■■

```
# Test config syntax
sudo rsyslogd -N1

# Reload without restart
sudo systemctl reload rsyslog

# Live log monitor
sudo tail -f /var/log/syslog
sudo tail -f /var/log/remote/<hostname>/<service>.log

# Search logs
grep 'error' /var/log/remote/client1/syslog
journalctl -u rsyslog --since '1 hour ago'

# Check open port 514
ss -tulnp | grep 514
```

Quick Reference — All Services

Service	Package	Config File	Port(s)	Start Command
NFS Server	nfs-kernel-server	/etc/exports	2049/111	systemctl start nfs-kernel-server
DNS BIND9	bind9	/etc/bind/named.conf	53	systemctl start bind9
Samba	samba	/etc/samba/smb.conf	445/139	systemctl start smbd nmbd
SMTP	postfix	/etc/postfix/main.cf	25/587/465	systemctl start postfix
FTP	vsftpd	/etc/vsftpd.conf	21/20	systemctl start vsftpd
SFTP	openssh-server	/etc/ssh/sshd_config	22	systemctl restart sshd
RSyslog	rsyslog	/etc/rsyslog.conf	514	systemctl start rsyslog
AutoFS	autofs	/etc/auto.master	—	systemctl start autofs

Key Commands Summary

```
# Verify all services are running
systemctl status nfs-kernel-server bind9 smbd vsftpd postfix sshd rsyslog autofs

# Check listening ports
ss -tulnp | grep -E '21|22|25|53|111|139|445|514|2049|3128'

# UFW — allow all service ports at once
sudo ufw allow 21,22,25,53,80,443,445,514,2049/tcp
sudo ufw allow 53,514,111,2049/udp
sudo ufw reload
```